

Robot Velocity Kinematics and Control — Part II

Jacobian-Based Velocity Control

Topics covered in this part:

- Motivation: why Jacobians for control
 - Pose representation in SE(3)
 - Pose error and matrix logarithm
 - Kinematic control law $\dot{q} = -k J^+ \xi_{\text{error}}$
 - Stability analysis (Lyapunov)
 - Pseudoinverse and motor implementation
-

Motivation: Using the Jacobian for Control

We seek expressions that relate joint velocities to end-effector motion:

Component form

$$v_n^0 = J_v \dot{q}, \quad \omega_n^0 = J_\omega \dot{q}$$

where

$$\xi = \begin{bmatrix} v_n^0 \\ \omega_n^0 \end{bmatrix} \in \mathbb{R}^6$$

$$J = \begin{bmatrix} J_v \\ J_\omega \end{bmatrix} \in \mathbb{R}^{6 \times n}$$

Compact form

$$\xi = J(q) \dot{q}$$

Dimension check

$$\underbrace{\xi}_{6 \times 1} = \underbrace{J(q)}_{6 \times n} \underbrace{\dot{q}}_{n \times 1}$$

We use the geometric Jacobian to compute joint velocities $\dot{q}$ that drive the end-effector toward a desired pose. The twist ξ desired is shaped by a control law based on the pose error.

Kinematic Control Design — Structured Procedure

$$(\cdot)^\vee : \mathfrak{se}(3) \rightarrow \mathbb{R}^6$$

$$\hat{\xi} = \begin{bmatrix} 0 & -\omega_z & \omega_y & v_x \\ \omega_z & 0 & -\omega_x & v_y \\ -\omega_y & \omega_x & 0 & v_z \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\hat{\xi}^\vee = \begin{bmatrix} v_x \\ v_y \\ v_z \\ \omega_x \\ \omega_y \\ \omega_z \end{bmatrix}$$

Given the system $\dot{\xi} = J(q)\dot{q}$, we follow a six-step structured approach:

1 Identify the system and its control input

✓ covered in this lecture

System: $\dot{\xi} = J(q)\dot{q}$ — Control input: $\dot{q} \in \mathbb{R}^n$ (joint velocity commands sent to each motor)

2 Define the pose error function

✓ covered in this lecture

Construct $T_{\text{error}} = T_d \cdot T^{-1} \in \text{SE}(3)$ and extract $\xi_e = \text{vee}(\log(T_e)) \in \mathbb{R}^6$

3 Design the kinematic control law

✓ covered in this lecture

Propose $\dot{q} = -k J^+(q) \xi_{\text{error}}$ so that joint velocities actively reduce the pose error

4 Analyze the closed-loop behavior

✓ covered in this lecture

Use Lyapunov analysis to confirm convergence of $\xi_e \rightarrow 0$ as $t \rightarrow \infty$

5 Simulate the system response

Integrate the closed-loop ODE numerically and verify that the robot reaches the desired pose

6 Implement the control on hardware

Send commands to each motor inner velocity PID loop at each control timestep

The Vee Operator $\mathbf{V} : \mathfrak{se}(3) \rightarrow \mathbb{R}^6$

$$\boldsymbol{\omega} = \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix}$$

$$[\boldsymbol{\omega}]_{\times} = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix}$$

The vee operator extracts the six independent components from a 4×4 matrix in $\mathfrak{se}(3)$ into a twist vector:

Definition

$$\hat{\xi} = \begin{bmatrix} [\boldsymbol{\omega}]_{\times} & \mathbf{v} \\ \mathbf{0} & 0 \end{bmatrix} \Rightarrow \xi = \begin{bmatrix} \mathbf{v} \\ \boldsymbol{\omega} \end{bmatrix} \in \mathbb{R}^6$$

Numerical Example

Given the $\mathfrak{se}(3)$ matrix $\hat{\xi}$:

$$\hat{\xi} = \begin{bmatrix} 0 & -\omega_z & \omega_y & v_x \\ \omega_z & 0 & -\omega_x & v_y \\ -\omega_y & \omega_x & 0 & v_z \\ 0 & 0 & 0 & 0 \end{bmatrix}$$



Applying $\mathbf{V}$ extracts:

$$\xi^{\mathbf{V}} = \begin{bmatrix} v_x \\ v_y \\ v_z \\ \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} \in \mathbb{R}^6$$

Reading:

$\mathbf{v} = [1, 4, 5]^T$ (last column, rows 1–3)

$\omega_z = 3$ (R_{21})

$\omega_y = -1$ (R_{13})

$\omega_x = -2$ (R_{32}):

$\boldsymbol{\omega} = [-1, 2, 3]^T$ (skew-sym.)

Pose Kinematics: From SO(3) to SE(3)

Rotation is a special case of full rigid-body motion. Understanding both structures is essential.

Property	Rotation $R(t) \in SO(3)$	Pose $T(t) \in SE(3)$
Variable	Rotation matrix R	$T = \begin{bmatrix} R & p \\ 0^\top & 1 \end{bmatrix} \in SE(3)$
Velocity	$\omega \in \mathbb{R}^3$	$\xi = \begin{bmatrix} v \\ \omega \end{bmatrix} \in \mathbb{R}^6$
Time deriv.	$\dot{R} = S(\omega) R$	$\dot{T} = T \hat{\xi}$
Space	SO(3) — rotation group	SE(3) — rigid motion group

Key insight: $\dot{R} = S(\omega) R$ is the special case of $\dot{T} = T \hat{\xi}$ when there is no translation. SE(3) unifies both rotation and translation into one framework.

Defining the Desired Pose

Rather than specifying an absolute target in the world frame, we define the **desired pose** relative to the current one:

Absolute specification (less flexible)

Fixed target in world frame — requires knowing the full current pose:

$$\mathbf{T}_{\text{desired}} = \text{SE3}(\mathbf{p}_{\text{absolute}})$$

Relative specification ✓ preferred

Desired pose relative to current end-effector pose:

$$\mathbf{T}_{\text{desired}} = \mathbf{T}_{\text{current}} \cdot \text{SE3.Trans}(\Delta x, \Delta y, \Delta z)$$

Benefits of the relative specification

- **Frame-independent:** Motion is defined relative to current pose — no global frame assumptions needed.
- **Reactive control:** Enables intuitive commands like "move 20 cm up" without knowing the full current pose.
- **Safer by design:** Avoids unexpected jumps if the robot is not in a known starting configuration.
- **Generalizable:** Can be reused regardless of the robot's initial state — ideal for visual servoing.

Pose Error Definition in SE(3)

To drive the robot toward a goal, we quantify the discrepancy between current and desired pose as a rigid-body transformation:

Pose error definition

$$\mathbf{T}_{\text{error}} = \mathbf{T}_d \cdot \mathbf{T}^{-1} \in SE(3)$$

$\mathbf{T}_d \in SE(3)$

Desired pose — where the robot should go

$\mathbf{T} \in SE(3)$

Current pose — where the robot is now

$\mathbf{T}_{\text{error}} \in SE(3)$

Pose error — transformation from current to desired

Explicit computation

$$\mathbf{R}_{\text{error}} = \mathbf{R}_d \cdot \mathbf{R}^\top$$

$$\mathbf{p}_{\text{error}} = \mathbf{p}_d - \mathbf{R}_d \mathbf{R}^\top \mathbf{p}$$

Inverse of $\mathbf{T}$

$$\mathbf{T}^{-1} = \begin{bmatrix} \mathbf{R}^\top & -\mathbf{R}^\top \mathbf{p} \\ \mathbf{0}^\top & 1 \end{bmatrix}$$

From Pose Error to Twist Error: The Matrix Logarithm

$T_e \in SE(3)$ is a matrix — but the control law needs a vector. We use the matrix logarithm + vee operator:

Key mapping

$$\xi_{\text{error}} = \text{vee}(\log(T_{\text{error}})) \in \mathbb{R}^6$$

Step 1 — Matrix logarithm $\log: SE(3) \rightarrow se(3)$

$$\log(T_{\text{error}}) = \begin{bmatrix} S(\omega) & \mathbf{v} \\ \mathbf{0}^\top & 0 \end{bmatrix} \in \mathfrak{se}(3)$$

The result lies in the Lie algebra $se(3)$ — a tangent space at the identity.

Step 2 — Vee operator $V: se(3) \rightarrow \mathbb{R}^6$

Extracts the 6 independent components:

- $\mathbf{v} \in \mathbb{R}^3$ — linear velocity component
- $\omega \in \mathbb{R}^3$ — angular velocity component

$$\xi_{\text{error}} = \text{vee}(\log(T_{\text{error}})) = \begin{bmatrix} \mathbf{v} \\ \omega \end{bmatrix} \in \mathbb{R}^6$$

Computing the Angular Error: Inverse Rodrigues Formulas

To evaluate $\log(T_e)$, we need the angular velocity $\omega = \theta \hat{u}$ from the rotation matrix $R_e \in SO(3)$:

Given $R_e \in SO(3)$, extract θ and $\hat{u}$:

1 Rotation angle θ

$$\theta = \arccos\left(\frac{\text{tr}(R) - 1}{2}\right)$$

2 Unit rotation axis $\hat{u}$ (for $\theta \neq 0, \pi$)

$$\hat{u} = \frac{1}{2 \sin \theta} \begin{bmatrix} R_{32} - R_{23} \\ R_{13} - R_{31} \\ R_{21} - R_{12} \end{bmatrix}$$

Special cases

$\theta \approx 0$

$R \approx I \rightarrow$ no rotation. Set $\omega = 0$.

$\theta \approx \pi$

Numerically unstable. Derive $\hat{u}$ from largest diagonal element of R .

Angular error vector

$$\omega = \theta \hat{u} \in \mathbb{R}^3$$

This is the ω extracted from $\log(R_e)$ and placed into ξ_e .

The Kinematic Control Law

Once the **pose error** has been mapped to a **twist** ξ_{error} , we close the loop with a proportional velocity controller:

Control Law

$$\dot{q} = -k J^\dagger(q) \xi_{\text{error}}$$

Parameters

$\dot{q} \in \mathbb{R}^n$	Joint velocity command — sent to each motor's inner velocity loop
$J^\dagger(q) \in \mathbb{R}^{n \times 6}$	Pseudoinverse of the geometric Jacobian — maps twist space $\rightarrow$ joint space
$\xi_{\text{error}} \in \mathbb{R}^6$	Spatial twist error — encodes how much translation and rotation remain
$k > 0$	Control gain — larger k gives faster convergence but risks oscillation

*Implementation note: $\dot{q}$ computed here is a **virtual control input**. It is sent as a reference to the inner velocity PID loop of each motor, which tracks it at a much faster rate. This outer loop operates purely at the kinematic level.*

Closed-Loop Analysis — Does the System Converge?

Substituting the control law into the system equation gives the closed-loop dynamics:

Closed-loop equation

$$\dot{\xi} = J(q) \left(-k J^\dagger(q) \xi_{\text{error}} \right) = -k J(q) J^\dagger(q) \xi_{\text{error}}$$

When $J(q)$ has full row rank (non-singular configuration):

$$J(q) J^\dagger(q) = \mathbf{I}_6 \quad (\text{when } J \text{ has full row rank})$$



$$\dot{\xi} = -k \xi_{\text{error}}$$

Stability proof via Lyapunov analysis

Candidate function

$$V = \frac{1}{2} \|\xi_{\text{error}}\|^2 > 0$$

Positive definite — measures energy of error

Time derivative

$$\dot{V} = \xi_{\text{error}}^\top \dot{\xi}_{\text{error}} \leq -k \|\xi_{\text{error}}\|^2 < 0$$

Negative definite — energy decreases along all trajectories

Conclusion: $\xi_{\text{error}} \rightarrow 0$ exponentially. The control law globally asymptotically stabilizes the pose error to zero (away from singularities).

Implementation: Pseudoinverse and Motor Commands

The control law requires $J^\dagger(q)$, the Moore–Penrose pseudoinverse. Its form depends on the shape of J :

Square ($n = 6$)

J invertible

$$J^\dagger = J^\top (J J^\top)^{-1}$$

Use J^{-1} directly when non-singular

Right ($n < 6$)

Underdetermined

$$J^\dagger = J^\top (J J^\top)^{-1}$$

Minimize $\| \dot{q} \| - J^\dagger = J^\top (J J^\top)^{-1}$

Left ($n > 6$)

Overdetermined

$$J^\dagger = (J^\top J)^{-1} J^\top$$

Least-squares — $J^\dagger = (J^\top J)^{-1} J^\top$

General method: Singular Value Decomposition (SVD)

$$J = U \Sigma V^\top \Rightarrow J^\dagger = V \Sigma^\dagger U^\top$$

Complete control pipeline (per timestep):



Summary — Jacobian-Based Velocity Control

The complete pipeline from pose error to joint motor commands:

$$\mathbf{T}_{\text{error}} = \mathbf{T}_d \cdot \mathbf{T}^{-1} \in SE(3)$$

Pose error — rigid-body transformation from current to desired pose

$$\boldsymbol{\xi}_{\text{error}} = \text{vee}(\log(\mathbf{T}_{\text{error}})) \in \mathbb{R}^6$$

Twist error — obtained via matrix logarithm and vee operator

$$\dot{\mathbf{q}} = -k J^\dagger(\mathbf{q}) \boldsymbol{\xi}_{\text{error}}$$

Kinematic control law — proportional feedback in task space

$$\underbrace{\boldsymbol{\xi}}_{6 \times 1} = \underbrace{J(\mathbf{q})}_{6 \times n} \underbrace{\dot{\mathbf{q}}}_{n \times 1}$$

Dimension check — the Jacobian maps joint velocities to Cartesian twist

Summary — Jacobian-Based Velocity Control

The complete pipeline from pose error to joint motor commands:

$$\mathbf{T}_{\text{error}} = \mathbf{T}_d \cdot \mathbf{T}^{-1} \in SE(3)$$

Pose error — rigid-body transformation from current to desired pose

$$\xi_{\text{error}} = \text{vee}(\log(\mathbf{T}_{\text{error}})) \in \mathbb{R}^6$$

Twist error — obtained via matrix logarithm and vee operator

$$\dot{q} = -k J^\dagger(q) \xi_{\text{error}}$$

Kinematic control law — proportional feedback in task space

Key takeaways

- The Jacobian $J(q)$ connects joint space to Cartesian task space
- Lyapunov analysis guarantees exponential convergence away from singularities
- The pose error lives in $SE(3)$ and is linearized via the matrix logarithm
- In practice: $\dot{q} = -k J^\dagger \xi_{\text{error}}$ is sent to motor velocity controllers at every timestep

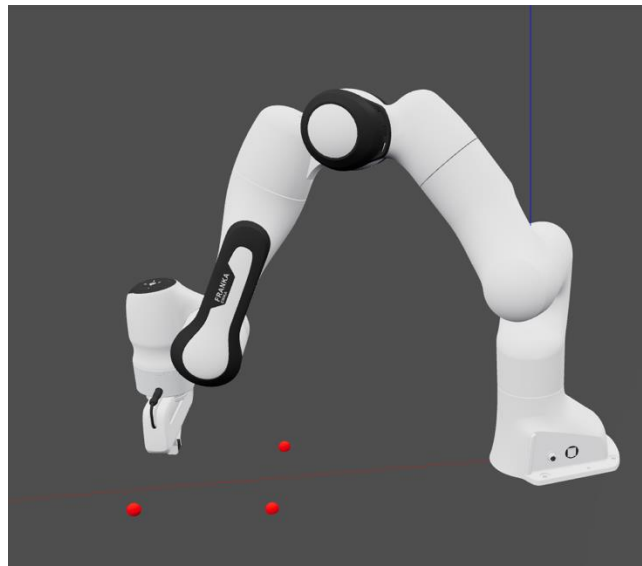
Simulation: Jacobian Kinematic Control

$$\dot{q} = -k J^{\dagger}(q) \xi_{\text{error}}$$

where:

- q = joint configuration
- $J(q)$ = robot Jacobian
- $J^{\dagger}(q)$ = pseudoinverse of the Jacobian
- ξ_{error} = end-effector pose error
- k = control gain (defines convergence speed)

This controller drives the robot end-effector toward a sequence of **target points in 3D space**.



Simulation Task

Run the following simulation from the course GitHub repository:

File name:

```
testCorke - KinematicControl4Points.py  
(or)  
testCorke - KinematicControl4Points -  
GoogleColab.ipynb
```

Install Required Python Libraries (VS Code / Terminal)

Run this command in your terminal before executing the simulation:

```
pip install roboticstoolbox-python spatialmath-python  
spatialgeometry swift-sim numpy matplotlib
```

Recommended (using a virtual environment):

Mac / Linux:

```
python -m venv venv  
source venv/bin/activate  
pip install roboticstoolbox-python spatialmath-python  
spatialgeometry swift-sim numpy matplotlib
```

Windows:

```
python -m venv venv  
venv\Scripts\activate  
pip install roboticstoolbox-python spatialmath-python spatialgeometry  
swift-sim numpy matplotlib
```


The code

```
panda.fkine(q)    → compute pose  
p_servo()         → compute task velocity  
jacobe()          → compute Jacobian  
pinv(J) @ v       → compute joint velocity  
env.step(dt)      → move robot
```

The controller is implemented here:

```
v, arrived = rtb.p_servo(panda.fkine(panda.q), Tep, gain=1, threshold=1e-3)
```

This line computes the **task-space velocity command**:

1. compares current pose `panda.fkine(panda.q)`
2. with desired pose `Tep`
3. returns velocity command `v`
4. checks if the robot reached the target (`arrived`)

Conceptually:

$$v = -k \xi_{error}$$

Jacobian-Based Joint Velocity Computation

Then this line converts task velocity into **joint velocities**:

```
panda.qd = np.linalg.pinv(panda.jacobe(panda.q)) @ v
```

This implements:

$$\dot{q} = J^{\dagger}(q) v$$

So:

- `jacobe()` → end-effector Jacobian
- `pinv()` → pseudoinverse
- `result` → joint velocity command

This is the **core kinematic controller**

Coding Tasks

Task 1 — Change the control speed

Modify the controller gain:

```
gain=1
```

Try different values:

```
gain = 0.5  
gain = 2  
gain = 5
```

Observe how the convergence speed changes.

Question:

Does the robot move faster? Does it become unstable?

Task 2 — Modify the target trajectory

Change the waypoint positions:

```
sm.SE3.Trans(-0.1, -0.1, 0.40)
```

Example:

```
sm.SE3.Trans(-0.2, 0.0, 0.30)
```

Try creating:

- a larger square
- a triangle
- a vertical motion trajectory

Goal: Understand how task-space targets affect robot motion.